



Module 5-bis : La Programmation Orientée Objet (POO) pour Cypress

Objectif du module : À la fin de ce module, l'étudiant saura :

- Comprendre le concept de **Classe** et d'**Objet** (le plan vs la construction).
- Utiliser le mot-clé **this** pour accéder aux données internes.
- Maîtriser l'**Export/Import** pour l'organisation professionnelle.
- Exécuter du JavaScript pur avec **Node.js** pour valider sa logique.



Architecture et Organisation du Module

Nous respectons la nomenclature avec tirets pour rester cohérent avec le reste de la formation.

Structure à reproduire dans votre VS Code :

Plaintext

```
cypress-formation/
├── javascript/
│   ├── module-1/
│   │   └── cours_classe.js
│   └── module-5-bis-poo/
│       ├── cours_classe.js
│       ├── exercice_poo.js
│       └── cypress/
│           └── e2e/
│               ├── module-5-requetes-http/
│               ├── module-5-bis-poo-tests/
│               └── intro_classes.cy.js
└── Node)
```

Bases de JS
<-- NOUVEAU : Pratique du JS pur
Notes et tests (exécutés avec
Vos exercices à réaliser
Module précédent
<-- NOUVEAU : Application Cypress
Test pour vérifier l'intégration

1. La Classe : Le "Moule à Gâteau" 🍰

Analogie : La **Classe** est un moule (le plan). L'**Objet** est le gâteau (le résultat réel). On peut créer autant de gâteaux que l'on veut avec un seul moule.

Code à tester dans javascript/module-5-bis-poo/cours_classe.js :

JavaScript

```
// 1. Définition du moule (La Classe)
class PageConnexion {
  url = "/login";
  champEmail = "#email";

  saisirEmail(email) {
    // On utilise console.log pour voir le résultat dans le terminal
    console.log("Action : Saisie de l'email -> " + email);
  }
}
```

```
// 2. Création de l'objet (L'Instance) avec "new"
const loginPage = new PageConnexion();

// 3. Utilisation des propriétés et méthodes
console.log("URL de la page :", loginPage.url);
loginPage.saisirEmail("etudiant@test.fr");
```

Instructions pour l'étudiant : Pour voir le résultat, ouvrez votre terminal et tapez :

```
node javascript/module-5-bis-poo/cours_classe.js
```

2. Le mot-clé `this` : "Mon propre..." 🙌

Analogie : `this` permet à une fonction de dire : "Regarde la variable qui est juste au-dessus de moi, dans la même boîte".

Exemple :

JavaScript

```
class Robot {
  nom = "Cyp-Bot";

  direBonjour() {
    console.log("Bonjour, je s'appelle " + this.nom);
  }
}
```

3. Export et Import (Préparer le Module 6) 📦

Dans Cypress, on sépare les fichiers pour ne pas avoir un code "fourre-tout".

- **Fichier A (page.js) :** On définit la classe et on l'exporte : `export default new Page();`
 - **Fichier B (test.cy.js) :** On l'importe : `import page from '../Page';`
-

🧩 Exercices pratiques

Exercice 1 : La Classe "Voiture" (Dossier javascript/)

(Fichier : `javascript/module-5-bis-poo/exercice_poo.js`)

1. Créez une classe `Voiture`.
2. Ajoutez une propriété `marque`.
3. Ajoutez une méthode `afficherMarque()` qui utilise `this.marque`.
4. Instanciez la voiture et lancez le fichier avec `node`.

Exercice 2 : Structure de Page (Dossier cypress/)

(Fichier : cypress/e2e/module-5-bis-poo-tests/intro_classes.cy.js)

1. Créez une classe `Header` directement dans le fichier de test.
2. Ajoutez-y une propriété `boutonProfil = ".btn-user"`.
3. Ajoutez une méthode `cliquerProfil()` qui fait un `cy.get(this.boutonProfil).click()`.
4. Utilisez-la dans un `it()`.



Correction des exercices

Correction 1 (Terminal Node) :

JavaScript

```
class Voiture {
  marque = "Tesla";
  afficherMarque() {
    console.log("Marque : " + this.marque);
  }
}
const maVoiture = new Voiture();
maVoiture.afficherMarque();
```

Correction 2 (Cypress Test Runner) :

JavaScript

```
class Header {
  boutonProfil = ".btn-user";
  cliquerProfil() {
    cy.get(this.boutonProfil).click();
  }
}
const header = new Header();

describe('Validation POO Cypress', () => {
  it('doit utiliser la méthode de la classe Header', () => {
    // Note : le test échouera si .btn-user n'existe pas,
    // mais la structure est correcte !
    header.cliquerProfil();
  });
});
```